Vibe Coding with the Pi Agent Loop

Ten Challenge Problems

An HPC-flavored curriculum in agentic loops

A graduated curriculum. Each problem introduces *exactly one* new agentic method on top of the previous one, so by Problem 10 the audience has *assembled* a self-improving multi-agent system out of primitives they met one at a time. HPC / science framing throughout so the examples land with the room.

0 Primitives (the five-minute cold open)

Pi invocations you'll reuse:

Form	What it is
`pi`	interactive REPL
`pi "..."`	one-shot, prints, exits
`pi -p "..."`	**print** / **headless** mode — the workhorse for scripted loops
`pi --json "..."`	structured output you can parse (use sparingly)
`pi -c` / `pi -r`	continue last / browse past sessions
`pi --fork <id>`	branch a session — the substrate for tournaments
`~/.pi/agent/skills/<name>/SKILL.md`	a self-installed, reusable skill
`~/.pi/agent/memory.md`	persistent lessons across runs

The Loop Anatomy (from `pi-loop-designer` — internalize this; every problem from #3 on is just these seven slots filled differently):

> **Objective · State · Iteration prompt · Verification · Reflection · Stop rules · Next prompt**

The single most important point of the whole tutorial: *stop prompting one task at a time. Design the loop; let the loop prompt the agent.* Verification is what makes a loop a loop — without a machine-checkable signal you just have a chatbot.

Problem 1 — Single-shot utility script

Teaches. a tight spec *is* the program. Precision in, artifact out.

Problem. Turn a `sacct` CSV dump into per-user GPU-hours.

```
pi -p 'Write slurm_gpu_hours.py. Input: a sacct CSV with columns
JobID,User,Elapsed,AllocTRES (TRES contains gres/gpu=N). Output: a table of
User → total GPU-hours, sorted descending. Parse Elapsed as [DD-]HH:MM:SS.
Handle missing gpu allocations as 0. stdlib only. Print a usage line if no arg.'
```

New idea. the entire "loop" here is *you reading the file*. Establishes the baseline so the room feels the upgrade in #2.

Problem 2 — Generate *with* its own verification (one shot)

Teaches. bundle the oracle with the artifact. Never ship code without the thing that checks it.

Problem. A roofline arithmetic-intensity helper, plus its own tests, run before you trust it.

```
pi -p 'Write roofline.py with arithmetic_intensity(flops, bytes_moved) and
attainable_gflops(ai, peak_gflops, mem_bw_gbs). THEN write test_roofline.py
covering: AI of a SAXPY, AI of a GEMM, the ridge point where compute- and
memory-bound cross, and div-by-zero. Run pytest -q yourself and only report
done if it is green. If red, fix and rerun before finishing.'
```

New idea. the agent runs its own check inside a single turn. This is the seed of the loop — we're about to externalize that check and let it run N times.

Problem 3 — Iterate-until-green (the canonical loop)

Teaches. the verification-gated loop. Tests are the stop rule; the shell is the loop runner.

Problem. Implement a 5-point 2D stencil (`src/stencil.py`) that passes a provided `test_stencil.py` (correctness vs a reference + a boundary case). You give the tests; the agent earns green.

```
# objective: pass tests | state: src/ + pytest output | verify: pytest exit code
# stop: green OR 8 attempts
for i in $(seq 1 8); do
  if pytest -q; then echo "OK: green in $i"; break; fi
  pi -p "pytest is failing. Read src/stencil.py and the failures below, make the
  smallest fix, do not edit the tests. Failures:
  $(pytest -q 2>&1 | tail -n 40)"
done
```

New idea. the loop lives in bash; **the verifier is pytest's exit code, not the model's opinion.** Drive verdicts off exit codes / files, never off prose. Off-the-shelf equivalent: `pi install npm:pi-loop-designer` — but build it by hand first so they see the bones.

Problem 4 — Differential / oracle-driven loop

Teaches. when there's no test suite, *manufacture* one from a slow-but-correct reference.

Problem. You have an obviously-correct `reference_softmax` (naive, loops, log-sum-exp). Write a fast vectorized `fast_softmax` and loop until it matches the oracle on 1000 random tensors to 1e-6.

```
# verify: a harness that diffs fast vs reference and exits nonzero on mismatch
cat > check.py <<'PY'
import numpy as np, sys
from reference import reference_softmax
```

```
from fast import fast_softmax
rng = np.random.default_rng(0)
for _ in range(1000):
    x = rng.standard_normal((rng.integers(1,64), rng.integers(1,512))) * rng.integers(1,50)
    if not np.allclose(fast_softmax(x), reference_softmax(x), atol=1e-6):
        print("MISMATCH"); sys.exit(1)
print("OK")
PY
for i in $(seq 1 6); do
  python check.py && { echo "OK: matches oracle"; break; }
  pi -p "fast.py disagrees with the reference oracle. Fix fast_softmax for
  numerical stability (think overflow on large logits). Don't touch reference.py.
  $(python check.py 2>&1 | tail -n 5)"
done
```

> **New idea.** **differential testing.** The oracle is the spec. This is the HPC optimizer's safety net — you go fast, the slow version keeps you honest.

Problem 5 — Tool-augmented loop (real-world feedback signal)

Teaches. verification doesn't have to be pass/fail. It can be a *measurement*. The loop optimizes against a profiler.

Problem. Speed up a memory-bound kernel. The loop's signal is wall-time + cache behavior from `perf stat`, and the stop rule is "throughput target hit OR no improvement for 3 rounds."

```
best=0
stale=0
for i in $(seq 1 12); do
  gflops=$(python bench.py --report gflops)          # harness prints a number
  echo "round $i: ${gflops} GFLOP/s (best ${best})"
  awk "BEGIN{exit !($gflops > 28)}" && { echo "HIT: target reached"; break; }
  awk "BEGIN{exit !($gflops > $best)}" && { best=$gflops; stale=0; } || stale=$((stale+1))
  [ $stale -ge 3 ] && { echo "PLATEAU"; break; }
  pi -p "Current kernel does ${gflops} GFLOP/s. Here is perf stat (cache-misses,
  bandwidth) and the hot loop. Propose ONE change (blocking, vectorization, layout)
  and apply it. Keep correctness vs check.py.
  $(perf stat -e cache-misses,cache-references python bench.py 2>&1 | tail -n 20)"
done
```

> **New idea.** **the environment is the grader.** A profiler, a benchmark, a linter, a simulator — any measurable signal can close the loop. Plus the first real **plateau stop rule**.

Problem 6 — Generator + Critic (role separation, fresh eyes)

Teaches. split *making* from *judging*. A critic with fresh context catches what the author is blind to (the `pi-review-loop` pattern).

Problem. Write an OpenMP-style parallel reduction (`reduce.py` w/ threads). The hazard is a data race / non-associative FP reduction. A separate critic pass hunts for it.

```
for i in $(seq 1 5); do
  # GENERATOR
  pi -p "Implement/improve reduce.py: a parallel sum over a large array.
  Address any open items in CRITIQUE.md if it exists."

  # CRITIC -- fresh session (no author bias), structured output, gates the loop
  pi -p --fork-fresh "You are a hostile reviewer. Inspect reduce.py ONLY for:
  data races, non-deterministic FP ordering, false sharing, off-by-one on the
  tail. Write findings to CRITIQUE.md as a checklist. If and only if there are
  zero real issues, write exactly NO_ISSUES on the first line."

  head -n1 CRITIQUE.md | grep -q NO_ISSUES && { echo "OK: critic satisfied"; break; }
done
```

New idea. **adversarial self-review with a clean context window.** Generator and critic are the same model, different roles and *different state* — that's why it works. `-fork-fresh` is shown conceptually; in practice fork the session or run the critic with a stripped context.

Problem 7 — Plan → decompose → sub-agent fan-out

Teaches. an orchestrator that splits work, spawns workers under **independent** context, and integrates. Parallelism + context isolation.

Problem. Build a GEMM benchmarking harness comparing three backends (NumPy, Torch, a hand kernel). One planner, three workers, one integrator.

```
# PLAN
pi -p 'Write PLAN.md decomposing a GEMM benchmark harness into independent
subtasks: one adapter per backend (numpy, torch, custom) with a common
interface gemm(A,B)→C, plus a runner. List each subtask + its acceptance test.'

# FAN-OUT: one isolated worker per backend, in parallel, each in its own dir/context
for be in numpy torch custom; do
  ( mkdir -p work/$be && cd work/$be
    pi -p "Per ../../PLAN.md, implement ONLY the $be adapter to the shared
    interface and a unit test proving it matches numpy on a 256x256 GEMM." ) &
done
wait

# INTEGRATE
pi -p 'All three adapters exist under work/*/. Wire them into bench.py behind the
common interface, run all unit tests, then produce a timing table. Fix integration
gaps only.'
```

New idea. **delegation with fresh per-worker context** (each worker doesn't drown in the others' detail) and **fan-out parallelism**. The orchestration is at the shell layer — honest and portable. In-agent alternative: Pi's `orchestrate` skill / delegation tool.

Problem 8 — Reflection / self-improvement loop

Teaches. the loop edits *itself*. On failure it doesn't just retry — it extracts a lesson and rewrites its own strategy / skill so it stops repeating mistakes. Pi's self-extensibility is the star.

Problem. Get a flaky numerical ODE integrator to converge to a known analytic solution. The trap is accumulation error / dtype / summation order. The loop must *learn* the fix, not stumble onto it.

```
for i in $(seq 1 8); do
  python converge_check.py && { echo "OK: converged"; break; }
  pi -p "integrator.py fails the convergence check (see error below).
  1) Fix it.
  2) Append the ROOT-CAUSE LESSON (one line, e.g. 'use Kahan summation for the
     accumulator') to ~/.pi/agent/memory.md so future runs start smarter.
  3) If this class of bug recurs, promote the lesson into a skill at
     ~/.pi/agent/skills/numerical-stability/SKILL.md.
  $(python converge_check.py 2>&1 | tail -n 15)"
done
```

New idea. the loop improves the loop. Memory + self-authored skills mean attempt $k{+}1$ is primed with the distilled lessons of $1 \ldots k$. This is meta-prompting that persists across runs — exactly what Pi was built for ("ask the agent to extend itself").

Problem 9 — Tournament (best-of-N with a fitness function)

Teaches. population search. Generate many *independent* candidates, score them all by an objective fitness, keep the champion. Variance is a feature, not a bug.

Problem. Implement an FFT-based 1D convolution. Spawn 8 independent attempts (forked sessions, varied temperature / model), score each by `fitness = correct ? gflops : 0`, crown the best.

```
mkdir -p arena
for n in $(seq 1 8); do
  ( cd arena && mkdir -p cand_$n && cd cand_$n
    pi -p "Implement conv1d_fft(signal, kernel) in conv.py. Optimize for speed
    on length-1e6 inputs. It must match numpy.convolve within 1e-5." ) &
done
wait

# SCORE every candidate with the SAME harness; champion = max fitness
best=""; bestf=0
for d in arena/cand_*; do
  f=$(python score.py "$d/conv.py")     # prints 0 if wrong, else GFLOP/s
  echo "$d → $f"
  awk "BEGIN{exit !($f > $bestf)}" && { bestf=$f; best=$d; }
done
echo "CHAMPION: $best @ $bestf GFLOP/s"
cp "$best/conv.py" conv.py
```

New idea. parallel best-of-N against a single fitness function. Independent samples beat one long chain when the search space is rough. Forked sessions = cheap diversity.

Problem 10 — Capstone: Tournament + Meta multi-agent

Teaches. *compose everything.* A meta-orchestrator runs rounds; each round is a tournament of mutated workers (P9); a critic ensemble prunes (P6); a benchmark assigns fitness (P5); a reflection step rewrites the mutation strategy for the next round (P8); plateau / budget ends it (P5). This is the AlphaEvolve shape, built from parts they've already seen.

Problem. *Evolve* an optimized SpMV (sparse matrix–vector) kernel for a given sparsity pattern — no human writes the final kernel, the system discovers it.

```
# --- meta loop: rounds of {mutate → critique → benchmark → select → reflect} ---
seed=spmv_seed.py; cp $seed champion.py; bestf=$(python score.py champion.py)
echo "round 0 champion fitness: $bestf"

for round in $(seq 1 6); do
  # 1) TOURNAMENT: spawn K mutated workers off the current champion (fresh contexts)
  for k in $(seq 1 6); do
    ( mkdir -p gen/$round/$k
      pi -p "Mutate champion.py (CSR SpMV) using ONE idea from STRATEGY.md:
      $(cat gen/$round/$k 2>/dev/null). Apply a single optimization (blocking,
      SIMD, format change, prefetch) → gen/$round/$k/spmv.py. Keep it correct
      vs check.py." ) &
  done; wait

  # 2) CRITIC ENSEMBLE: cheap fresh-eyes pass drops obviously-broken candidates
  for c in gen/$round/*/spmv.py; do
    pi -p --fork-fresh "Sanity-review $c for correctness + UB. Write KEEP or DROP
    to ${c%.py}.verdict on line 1."
  done

  # 3) BENCHMARK → fitness on survivors; 4) SELECT champion of the round
  for c in gen/$round/*/spmv.py; do
    grep -q '^KEEP' "${c%.py}.verdict" || continue
    f=$(python score.py "$c"); echo "  $c → $f"
    awk "BEGIN{exit !($f > $bestf)}" && { bestf=$f; cp "$c" champion.py; }
  done

  # 5) REFLECT: champion rewrites the mutation strategy for the NEXT round
  pi -p "Current champion fitness ${bestf} GFLOP/s. Review which mutation ideas
  this round helped vs hurt (gen/$round/*) and REWRITE STRATEGY.md with the most
  promising directions to try next. Be specific to this sparsity pattern."

  echo "round $round champion fitness: $bestf"
  # 6) STOP: plateau handled by comparing bestf across rounds (omitted for brevity)
done
echo "EVOLVED champion → champion.py @ $bestf GFLOP/s"
```

New idea. a self-improving system, not a script. Workers (P7) compete in tournaments (P9), get judged (P6) on real measurements (P5), and the strategy itself evolves via reflection (P8) under explicit stop rules. Every piece was introduced earlier — the capstone is the *composition*.

How this maps to four hours

Block	Problems	Method introduced
Warm-up (30m)	1–2	spec → artifact, bundled verification
Core loop (60m)	3–5	iterate-until-green, oracle diff, profiler-in-the-loop
Roles & scale (60m)	6–7	generator / critic, plan → fan-out
Self-improvement (45m)	8–9	reflection + memory + skills, tournaments
Capstone build (45m)	10	compose all of the above live

Three lines to leave them with:

1. A loop without a machine-checkable verifier is just a chatbot.
2. Drive verdicts off exit codes and files, never off the model's prose.
3. Stop prompting one task at a time — design the loop, let the loop prompt the agent.